Automation mesh for VM environments

Thank you for your interest in Red Hat Ansible Automation Platform. Ansible Automation Platform is a commercial offering that helps teams manage complex multi-tier deployments by adding control, knowledge, and delegation to Ansible-powered environments.

This guide helps you to understand the requirements and processes behind setting up an automation mesh on a VM-based installation of Red Hat Ansible Automation Platform. This document has been updated to include information for the latest release of Ansible Automation Platform.

Providing feedback on Red Hat documentation

If you have a suggestion to improve this documentation, or find an error, you can contact technical support at https://access.redhat.com to open a request.

1. Planning for automation mesh in your VM-based Red Hat Ansible Automation Platform environment

The following topics contain information to help plan an automation mesh deployment in your VM-based Ansible Automation Platform environment. The subsequent sections explain the concepts that comprise automation mesh in addition to providing examples on how you can design automation mesh topologies. Simple to complex topology examples are included to illustrate the various ways you can deploy automation mesh.

1.1. About automation mesh

Automation mesh is an overlay network intended to ease the distribution of work across a large and dispersed collection of workers through nodes that establish peer-to-peer connections with each other using existing networks.

Red Hat Ansible Automation Platform 2 replaces Ansible Tower and isolated nodes with Ansible Automation Platform and automation hub. Ansible Automation Platform provides the control plane for automation through its UI, RESTful API, RBAC, workflows and CI/CD integration, while automation mesh can be used for setting up, discovering, changing or modifying the nodes that form the control and execution layers.

Automation mesh uses TLS encryption for communication, so traffic that traverses external networks (the internet or other) is encrypted in transit.

Automation mesh introduces:

- Dynamic cluster capacity that scales independently, enabling you to create, register, group, ungroup and deregister nodes with minimal downtime.
- Control and execution plane separation that enables you to scale playbook execution capacity independently from control plane capacity.
- Deployment choices that are resilient to latency, reconfigurable without outage, and that dynamically re-reroute to choose a different path when outages occur.
- Connectivity that includes bi-directional, multi-hopped mesh communication possibilities which are *Federal Information Processing Standards* (FIPS) compliant.

1.2. Control and execution planes

Automation mesh makes use of unique node types to create both the **control** and **execution** plane. Learn more about the control and execution plane and their node types before designing your automation mesh topology.

1.2.1. Control plane

The **control plane** consists of hybrid and control nodes. Instances in the control plane run persistent automation controller services such as the the web server and task dispatcher, in addition to project updates, and management jobs.

- **Hybrid nodes** - this is the default node type for control plane nodes, responsible for automation controller runtime functions like project updates, management jobs and `ansible-runner` task operations. Hybrid nodes are also used for automation execution.
- **Control nodes** - control nodes run project and inventory updates and system jobs, but not regular jobs. Execution capabilities are disabled on these nodes.

1.2.2. Execution plane

The **execution plane** consists of execution nodes that execute automation on behalf of the control plane and have no control functions. Hop nodes serve to communicate. Nodes in the **execution plane** only run user-space jobs, and may be geographically separated, with high latency, from the control plane.

- **Execution nodes** - Execution nodes run jobs under `ansible-runner` with `podman` isolation. This node type is similar to isolated nodes. This is the default node type for execution plane nodes.
- **Hop nodes** - similar to a jump host, hop nodes route traffic to other execution nodes. Hop nodes cannot execute automation.

1.2.3. Peers

Peer relationships define node-to-node connections. You can define peers within the `[automationcontroller]` and `[execution_nodes]` groups or using the `[automationcontroller:vars]` or `[execution_nodes:vars]` groups

Unresolved directive in platform/assembly-planning-mesh.adoc - include::platform/ref-defining-node-types.adoc[leveloffset=+2]

2. Setting up automation mesh

Configure the Ansible Automation Platform installer to set up automation mesh for your Ansible environment. Perform additional tasks to customize your installation, such as importing a Certificate Authority (CA) certificate.

2.1. automation mesh Installation

For an RPM or Containerized VM-based install of Ansible Automation Platform you use an installation program to set up automation mesh or to upgrade automation mesh. To provide Ansible Automation Platform with details about the nodes, groups, and peer relationships in your mesh network, you define them in an the `inventory` file in the installer bundle. For managed cloud, OpenShift, or operator environments, see Automation mesh for managed cloud or operator environments.

Additional resources

- RPM installation
- Automation mesh design patterns

2.2. Editing the Red Hat Ansible Automation Platform installer inventory file

You can use the Red Hat Ansible Automation Platform installer inventory file to specify your installation scenario.

2.2.1. For an RPM installation

Procedure

RPM installed package

```
$ cd /opt/ansible-automation-platform/installer/
```

Bundled installer

```
$ cd ansible-automation-platform-setup-bundle-<latest-version>
```

Online installer

```
$ cd ansible-automation-platform-setup-<latest-version>
```

2.2.2. For online installations

For online installations

```
cd <path-to-source-file>/ansible-automation-platform-containerized-setup-
<version_number>
```

For offline or bundled installations

```
cd <path-to-source-file/ansible-automation-platform-containerized-setup-bundle-
<version_number>-<arch_name>
```

1. Open the `inventory` file with a text editor.
2. Edit the `inventory` file parameters to specify your installation scenario.

   For containerized installation, see Configuring the inventory file

Additional resources

- Inventory file variables

2.3. Running the Red Hat Ansible Automation Platform installer setup script

Run the installation program setup script after updating the inventory file with all the required parameters to begin the installation and configuration of the platform.

RPM installer

Procedure

- Run the `setup.sh` script

  ```
  $ sudo ./setup.sh
  ```

  NOTE

  If you are running the setup as a non-root user with `sudo` privileges, you can use the following command:

  ```
  $ ANSIBLE_BECOME_METHOD='sudo'
  ANSIBLE_BECOME=True ./setup.sh
  ```

  Installation of Red Hat Ansible Automation Platform will begin.

  Next steps

  If you want to add additional nodes to your automation mesh after the initial setup, edit the

inventory file to add the new node, then rerun the `setup.sh` script.

Containerized installer

For information on installing containerized Ansible Automation Platform, see Installing containerized Ansible Automation Platform

2.4. Importing a Certificate Authority (CA) certificate

Additional resources

A Certificate Authority (CA) verifies and signs individual node certificates in an automation mesh environment. You can provide your own CA by specifying the path to the certificate and the private RSA key file in the `inventory` file of your Red Hat Ansible Automation Platform installer.

NOTE	The Ansible Automation Platform installation program generates a CA if you do not provide one.

Procedure

1. Open the `inventory` file for editing.
2. Add the `mesh_ca_keyfile` variable and specify the full path to the private RSA key (`.key`).
3. Add the `mesh_ca_certfile` variable and specify the full path to the CA certificate file (`.crt`).
4. Save the changes to the inventory file.

```
[all:vars]
mesh_ca_keyfile=/tmp/<mesh_CA>.key
mesh_ca_certfile=/tmp/<mesh_CA>.crt
```

5. With the CA files added to the inventory file, run the installation program to apply the CA.

Result

This process copies the CA to the to `/etc/receptor/tls/ca/` directory on each control and execution node on your mesh network.

2.4.1. Using custom signed certificates in automation mesh

Learn how to replace the default automation mesh installer-provided certificates with custom,organization-specific certificates.

NOTE	In the following procedure, replace `<FQDN/IP Address>` and <IP Address> with the Fully Qualified Domain Name (FQDN) or IP address of the node.

Procedure

1. Stop the receptor service on all automation controller and execution nodes.

```
# systemctl stop receptor
```

2. Generate a new Certificate Authority (CA) for your mesh network.
3. Replace "common ca" in the command below with the required common name.

```
# receptor --cert-init commonname="common ca" bits=4096
outcert=/etc/receptor/tls/ca/mesh-CA.crt outkey=/etc/receptor/tls/ca/mesh-CA.key
```

4. Generate a self-signed certificate request for each Controller and Execution Node.

```
# receptor --cert-makereq commonname=<FQDN/IP Address> bits=4096 nodeid=<FQDN/IP
Address> outreq=/etc/receptor/tls/<FQDN/IP Address>.csr
outkey=/etc/receptor/tls/<FQDN/IP Address>.key ipaddress=<IP Address> ipaddress=<IP
Address>
```

5. Sign the newly created certificates with your CA. Make sure you adjust the `notafter=` date to meet your organizational requirements. The example shown uses a date far in the future.

```
# receptor --cert-signreq verify=yes cacert=/etc/receptor/tls/ca/mesh-CA.crt
cakey=/etc/receptor/tls/ca/mesh-CA.key req=/etc/receptor/tls/<FQDN/IP Address>.csr
outcert=/etc/receptor/tls/<FQDN/IP Address>.crt notafter="2034-07-29T20:48:02Z"
```

6. Transfer the newly created and signed certificates to their nodes in the `/etc/receptor/tls/` directory.
7. The `mesh-CA.crt` file must be placed in `/etc/receptor/tls/ca`.
8. Ensure that the permissions and ownership of the certificate files are set correctly.
   - All files should be owned by receptor
   - All certificate files should have 0640 permissions.

     `# chown -R receptor: /etc/receptor; chmod 0640 /etc/receptor/tls/<FQDN/IP Address>.crt`

9. Start the receptor service on all Controller and Execution nodes.

   `# systemctl start receptor`

10. Verify the node status in the Ansible Automation Platform UI:
11. In the navigation panel, select **Automation Execution › Infrastructure › Instance Groups**
12. Select the default instance group, then go to the Instances tab.
13. Ensure that the status of all nodes is marked as Ready.
14. If any node is marked as **Unavailable**:
    a. Select the Unavailable node.
    b. Click **[Run Health Check]**.
    c. Refresh the page, and the node should now display as Ready.

2.4.2. Correcting multiple signed certificates

If `/etc/receptor/tls/ca/mesh-CA.crt` (for RPM-based installs) or `$HOME/aap/receptor/etc/mesh-CA.crt` (for containerized installs) contains more than 10 certificates, an error occurs.

Take the following steps on all automation controller and execution nodes within the Ansible Automation Platform environment.

Procedure

1. For an RPM-based install, make a backup of the `mesh-CA.crt` file:

```
cp -p /etc/receptor/tls/ca/mesh-CA.crt /etc/receptor/tls/ca/mesh-CA.crt-$(date +%F)
```

2. Delete everything past the first certificate within the `mesh-CA.crt` file, that is, keep only the first certificate that is present at the top of the file.
   a. Restart the receptor:

```
systemctl restart receptor
```

3. For a containerized install, make a backup of the `mesh-CA.crt` file:

```
cp -p $HOME/aap/receptor/etc/mesh-CA.crt $HOME/aap/receptor/etc/mesh-CA.crt-$(date +%F)
```

4. Delete everything past the first certificate within the `mesh-CA.crt` file, that is, keep only the first certificate that is present at the top of the file.
   a. Restart the receptor:

```
systemctl --user restart receptor
```

Additional resources

- System Requirements

3. Automation mesh design patterns

The automation mesh topologies in this section provide examples you can use to design a mesh deployment in your environment. Examples range from a simple, hydrid node deployment to a complex pattern that deploys numerous automation controller instances, employing several execution and hop nodes.

IMPORTANT

If you are creating a mesh similar to the following in a containerized environment:

- Replace the `node_type` variable with `receptor_type`
- Replace the `peers` variable with `receptor_peers`
- Replace inventory group names with explicit comma-separated lists of hostnames

The value of `receptor_peers` must be a comma-separated list of hostnames. Do not use inventory group names. For more information, see Adding

execution nodes.

Prerequisites

- You reviewed conceptual information on node types and relationships.

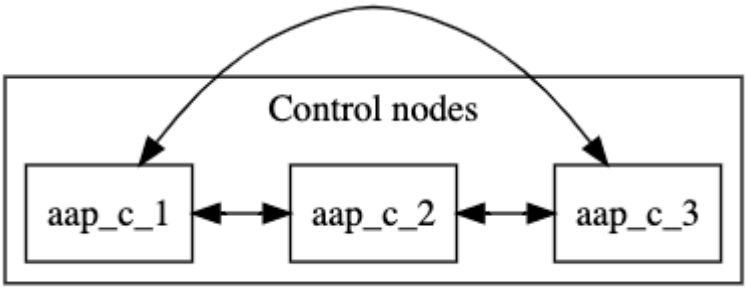
NOTE	The following examples include images that illustrate the mesh topology. The arrows in the images indicate the direction of peering. After peering is established, the connection between the nodes allows bidirectional communication.

3.1. Multiple hybrid nodes inventory file example

This example inventory file deploys a control plane consisting of multiple hybrid nodes. The nodes in the control plane are automatically peered to one another.

```
[automationcontroller]
aap_c_1.example.com
aap_c_2.example.com
aap_c_3.example.com
```

The following image displays the topology of this mesh network.



The default `node_type` for nodes in the control plane is `hybrid`. You can explicitly set the `node_type` of individual nodes to `hybrid` in the `[automationcontroller group]`:

```
[automationcontroller]
aap_c_1.example.com node_type=hybrid
aap_c_2.example.com node_type=hybrid
aap_c_3.example.com node_type=hybrid
```

Alternatively, you can set the `node_type` of all nodes in the `[automationcontroller]` group. When you add new nodes to the control plane they are automatically set to hybrid nodes.

```
[automationcontroller]
aap_c_1.example.com
aap_c_2.example.com
aap_c_3.example.com
```

```
[automationcontroller:vars]
node_type=hybrid
```

If you think that you might add control nodes to your control plane in future, it is better to define a separate group for the hybrid nodes, and set the `node_type` for the group:

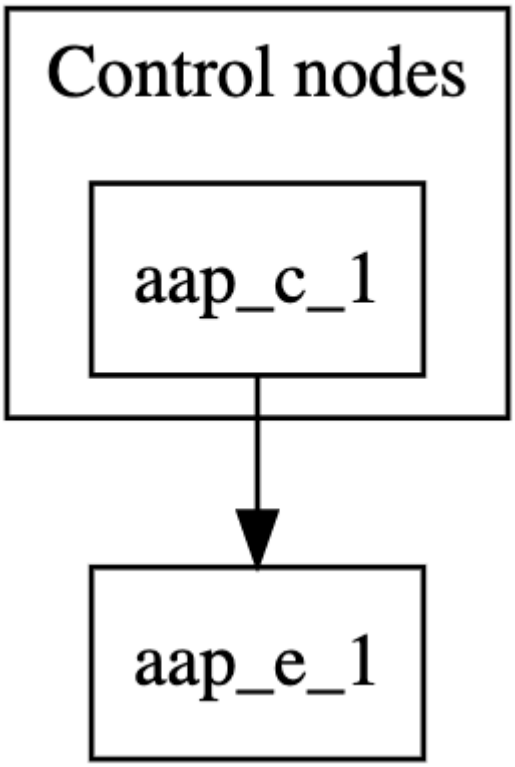
```
[automationcontroller]
aap_c_1.example.com
aap_c_2.example.com
aap_c_3.example.com

[hybrid_group]
aap_c_1.example.com
aap_c_2.example.com
aap_c_3.example.com

[hybrid_group:vars]
node_type=hybrid
```

3.1.1. Single node control plane with single execution node

This example inventory file deploys a single-node control plane and establishes a peer relationship to an execution node.

```
[automationcontroller]
aap_c_1.example.com

[automationcontroller:vars]
node_type=control
peers=execution_nodes

[execution_nodes]
aap_e_1.example.com
```

The following image displays the topology of this mesh network.



The `[automationcontroller]` stanza defines the control nodes. If you add a new node to the automationcontroller group, it will automatically peer with the `aap_c_1.example.com` node.

The `[automationcontroller:vars]` stanza sets the node type to `control` for all nodes in the control plane and defines how the nodes peer to the execution nodes:

- If you add a new node to the `execution_nodes` group, the control plane nodes automatically peer to it.
- If you add a new node to the `automationcontroller` group, the node type is set to `control`.

The `[execution_nodes]` stanza lists all the execution and hop nodes in the inventory. The default node type is `execution`. You can specify the node type for an individual node:

```
[execution_nodes]
aap_e_1.example.com node_type=execution
```

Alternatively, you can set the node_type of all execution nodes in the `[execution_nodes]` group. When you add new nodes to the group, they are automatically set to execution nodes.

```
[execution_nodes]
aap_e_1.example.com

[execution_nodes:vars]
node_type=execution
```

If you plan to add hop nodes to your inventory in future, it is better to define a separate group for

the execution nodes, and set the `node_type` for the group:

```
[execution_nodes]
aap_e_1.example.com

[local_execution_group]
aap_e_1.example.com

[local_execution_group:vars]
node_type=execution
```

3.2. Minimum resilient configuration

This example inventory file deploys a control plane consisting of two control nodes, and two execution nodes. All nodes in the control plane are automatically peered to one another. All nodes in the control plane are peered with all nodes in the `execution_nodes` group. This configuration is resilient because the execution nodes are reachable from all control nodes.

The capacity algorithm determines which control node is chosen when a job is launched. Refer to Automation controller capacity determination and job impact in Configuring automation execution for more information.

The following inventory file defines this configuration.

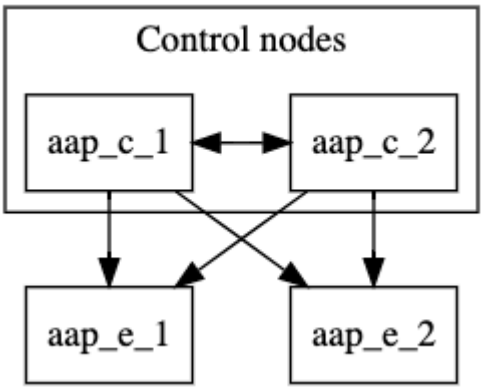
```
[automationcontroller]
aap_c_1.example.com
aap_c_2.example.com

[automationcontroller:vars]
node_type=control
peers=execution_nodes

[execution_nodes]
aap_e_1.example.com
aap_e_2.example.com
```

The `[automationcontroller]` stanza defines the control nodes. All nodes in the control plane are peered to one another. If you add a new node to the `automationcontroller` group, it will automatically peer with the original nodes.

The `[automationcontroller:vars]` stanza sets the node type to `control` for all nodes in the control plane and defines how the nodes peer to the execution nodes:

- If you add a new node to the `execution_nodes` group, the control plane nodes automatically peer to it.
- If you add a new node to the `automationcontroller` group, the node type is set to `control`.

The following image displays the topology of this mesh network.



3.3. Segregated local and remote execution configuration

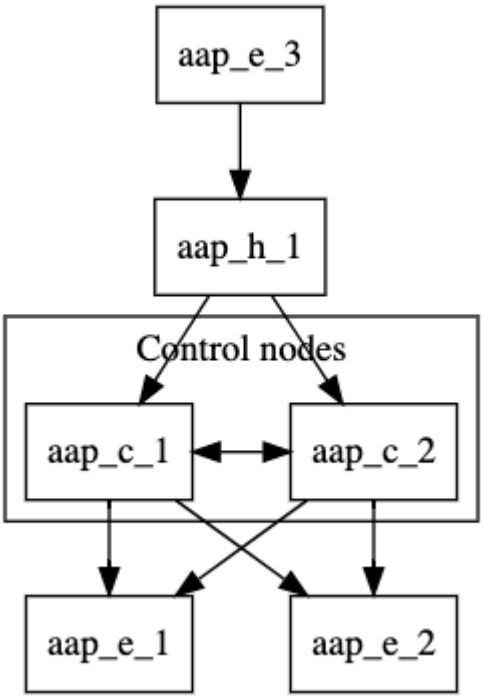
This configuration adds a hop node and a remote execution node to the resilient configuration. The remote execution node is reachable from the hop node.

You can use this setup if you are setting up execution nodes in a remote location, or if you need to run automation in a DMZ network.

```
[automationcontroller]
aap_c_1.example.com
aap_c_2.example.com

[automationcontroller:vars]
node_type=control
peers=instance_group_local

[execution_nodes]
aap_e_1.example.com
aap_e_2.example.com
aap_h_1.example.com node_type=hop
aap_e_3.example.com

[instance_group_local]
aap_e_1.example.com
aap_e_2.example.com

[hop]
aap_h_1.example.com

[hop:vars]
peers=automationcontroller

[instance_group_remote]
aap_e_3.example.com
```

```
[instance_group_remote:vars]
peers=hop
```

The following image displays the topology of this mesh network.



The `[automationcontroller:vars]` stanza sets the node types for all nodes in the control plane and defines how the control nodes peer to the local execution nodes:

- All nodes in the control plane are automatically peered to one another.
- All nodes in the control plane are peered with all local execution nodes.

If the name of a group of nodes begins with `instance_group_`, the installer recognises it as an instance group and adds it to the Ansible Automation Platform user interface.

3.3.1. Multi-hopped execution node

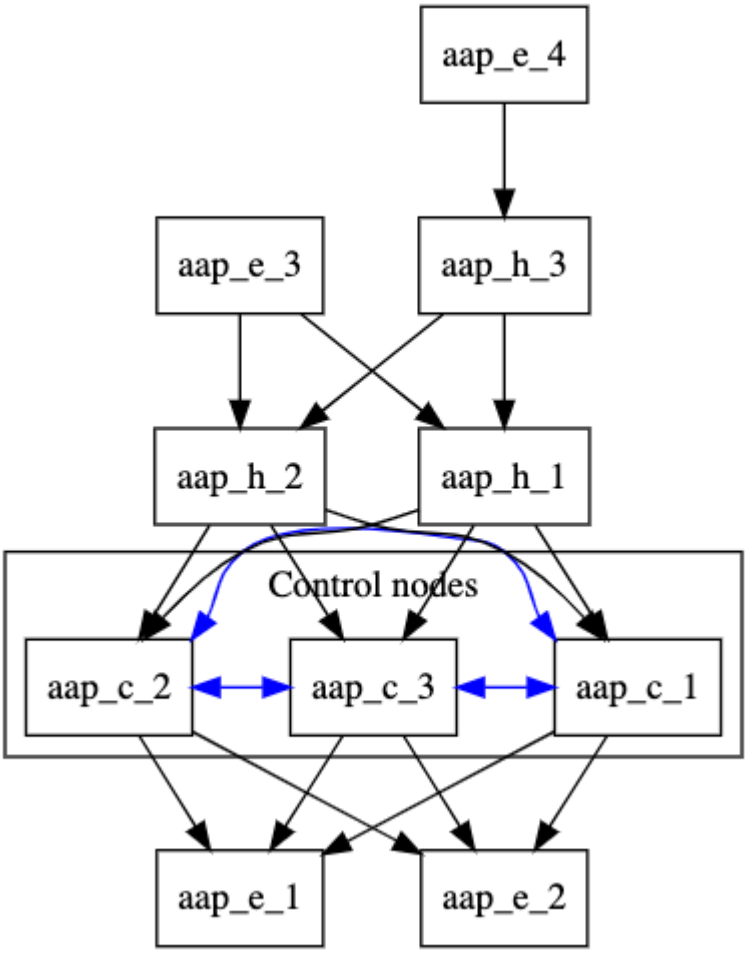
In this configuration, resilient controller nodes are peered with resilient local execution nodes. Resilient local hop nodes are peered with the controller nodes. A remote execution node and a remote hop node are peered with the local hop nodes.

You can use this setup if you need to run automation in a DMZ network from a remote network.

```
[automationcontroller]
aap_c_1.example.com
aap_c_2.example.com
aap_c_3.example.com

[automationcontroller:vars]
node_type=control
peers=instance_group_local
```

```
[execution_nodes]
aap_e_1.example.com
aap_e_2.example.com
aap_e_3.example.com
aap_e_4.example.com
aap_h_1.example.com node_type=hop
aap_h_2.example.com node_type=hop
aap_h_3.example.com node_type=hop

[instance_group_local]
aap_e_1.example.com
aap_e_2.example.com

[instance_group_remote]
aap_e_3.example.com

[instance_group_remote:vars]
peers=local_hop

[instance_group_multi_hop_remote]
aap_e_4.example.com

[instance_group_multi_hop_remote:vars]
peers=remote_multi_hop

[local_hop]
aap_h_1.example.com
aap_h_2.example.com

[local_hop:vars]
peers=automationcontroller

[remote_multi_hop]
aap_h_3 peers=local_hop
```

The following image displays the topology of this mesh network.



The `[automationcontroller:vars]` stanza sets the node types for all nodes in the control plane and defines how the control nodes peer to the local execution nodes:

- All nodes in the control plane are automatically peered to one another.
- All nodes in the control plane are peered with all local execution nodes.

The `[local_hop:vars]` stanza peers all nodes in the `[local_hop]` group with all the control nodes.

If the name of a group of nodes begins with `instance_group_`, the installer recognises it as an instance group and adds it to the Ansible Automation Platform user interface.

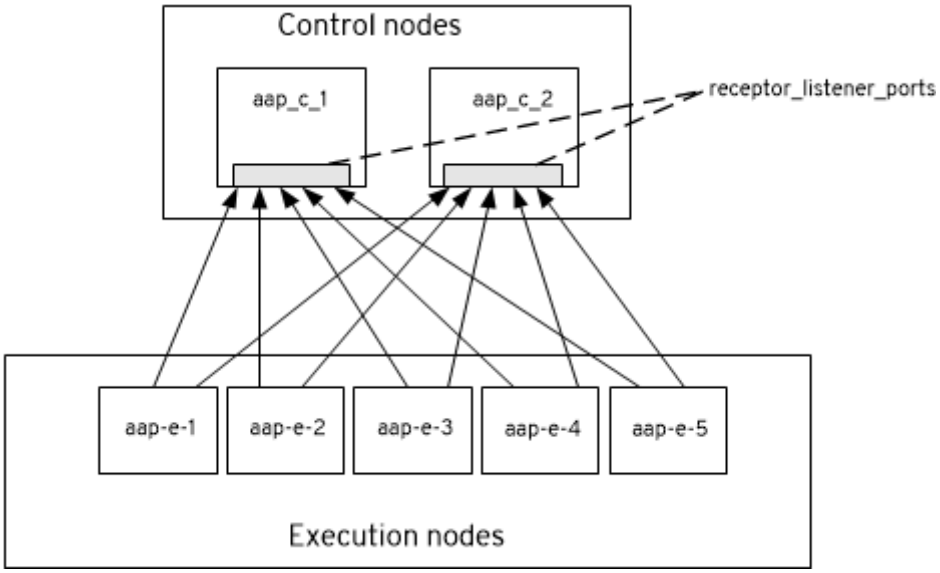
3.3.2. Outbound only connections to controller nodes

This example inventory file deploys a control plane consisting of two control nodes, and several execution nodes. Only outbound connections are allowed to the controller nodes All nodes in the 'execution_nodes' group are peered with all nodes in the controller plane.

```
[automationcontroller]
controller-[1:2].example.com

[execution_nodes]
execution-[1:5].example.com

[execution_nodes:vars]
# connection is established *from* the execution nodes *to* the automationcontroller
```

```
peers=automationcontroller
```

The following image displays the topology of this mesh network.



4. Deprovisioning individual nodes or groups

You can deprovision automation mesh nodes and instance groups by using the Ansible Automation Platform installer. Learn how to deprovision specific nodes or entire groups, with example inventory files.

IMPORTANT	Containerized Ansible Automation Platform deployments do not support the `node_state=deprovision` parameter. For containerized deployments, use the `awx-manage` command to deprovision nodes and groups. For more information, see Deprovisioning isolated nodes and Deprovisioning isolated instance groups.

4.1. Deprovisioning individual nodes using the installer

You can deprovision nodes from your automation mesh using the Ansible Automation Platform installer. Edit the `inventory` file to mark the nodes to deprovision, then run the installer. Running the installer also removes all configuration files and logs attached to the node.

NOTE	You can deprovision any of your inventory's hosts except for the first host specified in the `[automationcontroller]` group.

Procedure

- Append **`node_state=deprovision`** to nodes in the installer file you want to deprovision.

The following example inventory file deprovisions two nodes from an automation mesh configuration.

```
[automationcontroller]
126-addr.tatu.home ansible_host=192.168.111.126  node_type=control
121-addr.tatu.home ansible_host=192.168.111.121  node_type=hybrid
routable_hostname=121-addr.tatu.home
115-addr.tatu.home ansible_host=192.168.111.115  node_type=hybrid
node_state=deprovision

[automationcontroller:vars]
peers=connected_nodes

[execution_nodes]
110-addr.tatu.home ansible_host=192.168.111.110 receptor_listener_port=8928
108-addr.tatu.home ansible_host=192.168.111.108 receptor_listener_port=29182
node_state=deprovision
100-addr.tatu.home ansible_host=192.168.111.100 peers=110-addr.tatu.home
node_type=hop
```

4.2. Deprovisioning isolated nodes

You have the option to manually remove any isolated nodes using the `awx-manage` deprovisioning utility.

WARNING	Use the deprovisioning command to remove only isolated nodes that have not migrated to execution nodes. To deprovision execution nodes from your automation mesh architecture, use the Deprovisioning individual nodes using the installer method instead.

Procedure

1. Shut down the instance:

```
$ automation-controller-service stop
```

2. Run the deprovision command from another instance, replacing `host_name` with the name of the node as listed in the inventory file:

```
$ awx-manage deprovision_instance --hostname=<host_name>
```

4.3. Deprovisioning groups using the installer

You can deprovision entire groups from your automation mesh using the Ansible Automation Platform installer. Running the installer will remove all configuration files and logs attached to the nodes in the group.

NOTE	You can deprovision any hosts in your inventory except for the first host specified in the `[automationcontroller]` group.

Procedure

- Add `node_state=deprovision` to the [group:vars] associated with the group you want to deprovision, as in the following example:

  Group deprovision

  ```
  [execution_nodes]
  execution-node-1.example.com peers=execution-node-2.example.com
  execution-node-2.example.com peers=execution-node-3.example.com
  execution-node-3.example.com peers=execution-node-4.example.com
  execution-node-4.example.com peers=execution-node-5.example.com
  execution-node-5.example.com peers=execution-node-6.example.com
  execution-node-6.example.com peers=execution-node-7.example.com
  execution-node-7.example.com

  [execution_nodes:vars]
  node_state=deprovision
  ```

4.4. Deprovisioning isolated instance groups

You have the option to manually remove any isolated instance groups using the `awx-manage` deprovisioning utility.

WARNING	Use the deprovisioning command to only remove isolated instance groups. To deprovision instance groups from your automation mesh architecture, use the Deprovisioning groups using the installer method instead.

Procedure

- Run the following command, replacing `<name>` with the name of the instance group:

  ```
  $ awx-manage unregister_queue --queuename=<name>
  ```